L'objectif de cette fiche est de présenter le mécanisme d'**exceptions** en Java. Ce mécanisme permet de traiter de manière fine les erreurs et autres situations anormales en délégant ce traitement à des gestionnaires d'erreur appropriés.

Un exemple introductif

Ah, le pangolin... Quel animal merveilleux. Malgré son apparence ridicule, il permet, à lui seul, d'illustrer tous les concepts de la programmation orientée objet. Revenons donc à notre classe `Pangolin` définie dans les fiches précédentes, et dont voici un extrait :

```
class Pangolin {
    private String nom;
    private int nbEcailles;

    public Pangolin(String nom, int nbEcailles) {
        this.nom = nom;
        this.setNbEcailles(nbEcailles);
    }

    public void setNbEcailles(int nbEcailles) {
        this.nbEcailles = nbEcailles;
    }

    [...]
}
```

Même sans connaissance zoologique avancée, on peut aisément imaginer que le nombre d'écailles d'un pangolin ne peut être négatif. Pourtant, rien ne l'empêche dans ce code. Voici une première tentative naïve de modification de la méthode `setNbEcailles(int nb)` :

```
public void setNbEcailles(int nbEcailles) {
    if (nbEcailles >= 0) {
        this.nbEcailles = nb;
    } else {
        System.out.println("Pauvre pangolin... Il a un nombre d'écailles
négatif...");
        this.nbEcailles = 42;  // Un nombre d'écailles par défaut
    }
}
```

C'est un peu mieux : on ne met pas le pangolin dans un état incohérent et on prévient l'utilisateur s'il tente d'initialiser un pangolin avec un nombre d'écailles négatif. Mais ce n'est pas satisfaisant pour autant. D'une part, le traitement local de l'erreur consiste à fixer le nombre d'écailles à un nombre arbitraire sans se soucier de ce que veut vraiment l'utilisateur. D'autre part, on le prévient avec un simple message sur la sortie standard (ce qui lui fera une belle jambe s'il lance son programme sans console, par exemple lors d'une procédure de tests automatisés).

Essayons autre chose :

```
public void setNbEcailles(int nb) {
    if (nbEcailles >= 0) {
        this.nbEcailles = nb;
    } else {
        System.err.println("Tu as initialisé un pangolin avec un nombre
négatif d'écailles !!!!!\n"
```

```
                                + "Tu ne mérites rien d'autre que d'aller coder
en assembleur "
                                + "en écoutant le dernier album de Francis
Lalanne.\n"
                                + "Et d'ailleurs, pour bien te signifier mon
mépris profond "
                                + "je quitte le programme avec une erreur.");
            System.exit(1);
        }
}
```

Que penser de cette solution ? D'abord, on peut remarquer qu'il est probable que le développeur de cette méthode ait passé une sale journée, mais c'est un détail. D'autre part, c'est mieux que précédemment, car cette fois-ci, on signifie bien à l'utilisateur qu'il a fait n'importe quoi lors de l'appel de `setNbEcailles`, mais d'une manière probablement un peu extrême : on quitte le programme sur une erreur, sans sommation, et sans donner l'occasion à celui qui appelle `setNbEcailles` de comprendre ce qui se passe et de résoudre son erreur.

Comment s'en sortir ? Vous l'avez deviné, nous allons utiliser le mécanisme d'**exceptions**. Grosse surprise...

Exceptions : la théorie

Le mécanisme d'exceptions permet de gérer les situations anormales (comme des erreurs) qui peuvent survenir lors de l'exécution d'un programme. Sans ce mécanisme, on traite les cas exceptionnels par une série de conditionnelles, qui rendent le code peu lisible. Formellement :

Exception

Une exception est un événement anormal interrompant le flot d'exécution d'un programme. La gestion d'une exception repose sur trois phases :

- une exception est **levée** quand une erreur survient ;
- l'exception est ensuite **propagée**, c'est-à-dire que l'exécution séquentielle du programme est interrompue et le flot de contrôle transféré aux gestionnaires d'exception ;
- l'exception est **récupérée** par un gestionnaire d'exception, qui la traite. Ensuite, l'exécution reprend avec les instructions qui suivent le gestionnaire d'exception.

Attention

Si une exception n'est récupérée par aucun gestionnaire d'exception, alors le programme s'interrompt sur une erreur, affiche la nature de l'exception ainsi que toute la pile d'appel au moment où l'exception a été levée.

Afin d'illustrer une partie de ces concepts théoriques, considérons par exemple le programme suivant :

```java
public class TestException {
    public static void m1() {
        System.out.println("Début de m1()");
        m2();
        System.out.println("Fin de m1()");
    }

    public static void m2() {
        System.out.println("Début de m2()");
        m3();
        System.out.println("Fin de m2()");
    }

    public static void m3() {
        System.out.println("Début de m3()");
        int x = 0;
        int y = 1 / x;  // Très mauvais, ça : java.lang.ArithmeticException
        System.out.println("Fin de m3()");
    }

    public static void main(String[] args) {
        System.out.println("Début de la méthode principale");
        m1();
        System.out.println("Fin de la méthode principale");
    }
}
```

Lorsque l'on exécute ce programme, voici ce que l'on obtient :

```
Début de la méthode principale
Début de m1()
Début de m2()
Début de m3()
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at TestException.m3(TestException.java:17)
    at TestException.m2(TestException.java:10)
    at TestException.m1(TestException.java:4)
    at TestException.main(TestException.java:23)
```

Le comportement du programme est clair : un événement anormal (division par zéro) se produit à la ligne 17 du programme (ce qui correspond à la ligne 3 de la méthode `m3()`), provoquant la levée d'une exception de type `java.lang.ArithmeticException`. En conséquence le flot d'exécution normal est interrompu, ce qui explique que la dernière instruction de la méthode `m3()` n'est jamais exécutée. En l'absence de gestionnaire adéquat, l'exception est propagée vers la méthode appelante `m2()`, qui fait de même, et propage à son tour l'exception vers `m1()`, puis vers `main(String[] args)`. Lors de toute cette propagation, aucun gestionnaire adéquat n'ayant été rencontré, le programme s'interrompt en affichant la nature de l'exception, ainsi que la pile d'appels.

Les exceptions en Java

En Java, la notion d'exception est définie en pratique par la classe `Throwable` (que les anglicistes parmi vous auront traduit approximativement par « qui peut être lancé »). Tout objet étant une instance d'une classe qui hérite de `Throwable` sera considéré, en Java, comme une exception. Ainsi, par exemple, la classe `java.lang.ArithmeticException`, a pour arbre d'héritage : `java.lang.ArithmeticException` → `java.lang.RuntimeException` → `j`

`ava.lang.Exception` → `java.lang.Throwable` (puis `java.lang.Object`, bien entendu). Allez donc voir sur sa documentation si vous ne me croyez pas.

La hiérarchie des exceptions

Comme l'exemple de la classe `ArithmeticException` ci-dessus le suggère, il existe toute une hiérarchie d'exceptions dans la bibliothèque standard Java. Cette hiérarchie a pour racine la classe `Throwable`. Cette classe `Throwable` a deux sous-classes : `Error` et `Exception`. La classe `Error` est la classe des exceptions « graves », qui ne devraient pas être récupérées (erreurs système, plus de mémoire... etc.). La classe `Exception` correspond aux exceptions qu'on peut vouloir récupérer. Celle-ci a une sous-classe `RuntimeException`. Les exceptions que vous avez déjà rencontré précédemment (`ArithmeticException`, `IllegalArgumentException`) sont des sous-classes de `RuntimeException`.

Une partie de l'arbre représentant la hiérarchie des exceptions est représentée ci-dessous.

Exceptions sous contrôle / hors-contrôle

À l'instar du monde, les exceptions se divisent en deux catégories :

1. Les exceptions **hors-contrôle** (apparaissant en rouge sur le diagramme UML ci-dessus), qui sont les classes :
   - descendantes de `java.lang.Error` : ce sont typiquement des erreurs critiques qui ne peuvent pas être récupérées (plus de mémoire par exemple) ;
   - descendantes de `java.lang.RuntimeException` : il s'agit ici d'erreurs de programmation qui risquent de provoquer l'hilarité générale de vos collègues développeurs à la machine à café (par exemple la fameuse `NullPointerException`, grande classique des dîners mondains entre programmeurs).
2. Les exceptions **sous contrôle**, qui sont les classes dérivées de `java.lang.Exception`, mais pas de `java.lang.RuntimeException`. Elles correspondent typiquement aux cas d'erreurs que l'on a anticipés, et que l'on veut traiter. Ces exceptions doivent être récupérées et traitées dans le programme.

Gardez en tête ces deux catégories, nous y reviendrons un peu plus loin.

Définir sa propre exception

Même si la bibliothèque standard Java contient déjà un grand nombre d'exceptions, vous pouvez définir vos propres types d'exceptions. C'est même conseillé : ainsi, vous pourrez classifier finement les situations anormales auxquelles votre programme pourrait être confronté. Pour définir son propre type d'exception, rien de plus simple : il suffit de définir une classe fille de `Exception` (ou alors `Throwable`, même si l'usage veut que l'on sous-type plutôt `Exception`).

Revenons à notre pangolin par exemple et son nombre d'écailles négatif. Pour caractériser cette situation anormale, on pourrait tout-à-fait utiliser par exemple `IllegalArgumentException`, mais si l'on veut être plus précis dans la caractérisation de cette anomalie, on peut aussi définir notre propre type d'exception :

```
public class NbEcaillesIncorrectException extends Exception {
    public NbEcaillesIncorrectException(String message) {
        super(message);
    }
}
```

Et c'est tout... Dans le constructeur de la classe ci-dessus, le paramètre `message` correspond simplement au message qui sera affiché si l'erreur est lancée mais non rattrapée.

Lever, propager, rattraper une exception

Lever une exception

Nous avons vu ci-dessus que certaines instructions erronées pouvaient lever des exceptions, comme dans l'exemple de la méthode `m3()` déjà vu :

```
public static void m3() {
    System.out.println("Début de m3()");
    int x = 0;
    int y = 1 / x;  // Très mauvais, ça : java.lang.ArithmeticException
    System.out.println("Fin de m3()");
}
```

Dans cet exemple, l'instruction `int y = 1 / x;` provoque une exception qui correspond à une division par zéro : c'est une erreur de programmation (NB : d'ailleurs, il s'agit d'une exception hors-contrôle selon la classification définie ci-avant).

Mais nous pouvons également lever explicitement nos propres exceptions, à l'aide du mot-clef `throw`. Jugez plutôt :

```
public void setNbEcailles(int nbEcailles) {
    if (nbEcailles < 0) {
        throw new NbEcaillesIncorrectException("Nombre d'écailles
négatif !");
    }
    this.nbEcailles = nbEcailles;
}
```

Interlude : vous pouvez observer au passage qu'en Java, les exceptions sont **lancées** (`throw`), alors qu'en Python – par exemple –, les exceptions sont **levées** (`raise`)).

Contrainte *catch or specify*

Essayons de compiler notre classe Pangolin augmentée du code précédent qui lance une exception de type `NbEcaillesIncorrectException`. Voici ce que l'on obtient :

```
Pangolin.java:27: error: unreported exception NbEcaillesIncorrectException;
must be caught or declared to be thrown
```

```
            throw new NbEcaillesIncorrectException("Nombre d'écailles
négatif !");
            ^
1 error
```

Que signifie cette erreur ? Elle vient de la contrainte *catch or specify* de Java :

Contrainte *catch or specify*

En Java, toute exception **sous contrôle** lancée doit être :

- soit rattrapée et traitée localement dans la méthode dans laquelle elle a été lancée ;
- soit déclarée explicitement dans la signature de la méthode comme pouvant être lancée.

Insistons bien sur le fait que cette exigence ne concerne que les exceptions **sous contrôle**. Celles qui sont hors contrôle échappent donc à cette contrainte, ce qui explique le fait que tous les exemples précédents impliquant des exceptions compilaient sans erreur.

La contrainte *catch or specify* illustre simplement le fameux petit principe suivant de survie élémentaire en entreprise. Si vous faites une erreur (oui, tout le monde en fait, même vos enseignants de POO, aussi étonnant que ça puisse paraître), alors vous avez deux attitudes possibles :

- soit vous résolvez vous-même l'erreur, et dans ce cas-là, personne ne vous dira rien ;
- soit vous déléguez la résolution de l'erreur à la personne qui vous a mandaté pour faire ce job pourri : votre client, votre patron, etc. **Mais** (le gras indique que ce « mais » est important) vous ne pouvez faire ça que si au préalable vous aviez prévenu cette personne que ça pouvait tourner mal. Sinon, vous passez juste pour un incompétent (d'où, encore une fois, hilarité générale à la machine à café).

Nous allons maintenant voir comment implanter ces deux attitudes en Java.

(NB : il existe aussi en Java, comme dans la vraie vie, une attitude qui consiste à dissimuler l'erreur sous le tapis en espérant que personne ne la verra (voyez la note sur le gestionnaire attrape-tout plus loin). Vous serez priés de ne pas le faire.)

Résoudre localement une erreur : *try / catch*

Supposons que nous exécutions du code qui peut échouer en lançant une exception, et que nous voulions traiter cette exception localement. Alors, il faudra entourer ce code d'un bloc `try`, suivi immédiatement par un bloc `catch` contenant le code permettant de traiter l'exception. Un bloc `catch` doit être paramétré par le type de l'exception que ce bloc doit traiter.

Jugez plutôt :

```
public void setNbEcailles(int nbEcailles) {
    try {
        if (nbEcailles < 0) {
            throw new NbEcaillesIncorrectException("Nombre d'écailles
négatif !");
        }
        this.nbEcailles = nbEcailles;
    } catch (NbEcaillesIncorrectException e) {
        // Ici, on va traiter l'exception en question
        System.out.println(e);
        e.printStackTrace();
        System.exit(1);
    }
}
```

Dans l'exemple ci-dessus, si on appelle la méthode avec un paramètre négatif, une exception sera lancée par l'instruction `throw`. La ligne suivante (`this.nbEcailles = nbEcailles;`) ne sera donc pas exécutée, et le flot d'instructions sera donc directement dévié vers le gestionnaire d'exception, c'est-à-dire l'ensemble d'instructions du bloc `catch (NbEcaillesIncorrectException e)`. Ici, il n'y a qu'un seul bloc `catch`, mais on peut en mettre autant que l'on souhaite après un bloc `try`.

Vous me direz, dans ce cas, quel est l'intérêt d'alourdir le code avec des mots-clefs supplémentaires, alors que finalement, l'erreur est traitée localement, et qu'un simple `if` aurait suffi ? La question est excellente et je vous remercie de me l'avoir posée. Ici, l'intérêt est double :

1. Cette syntaxe permet de séparer très clairement le code exécuté lors d'un comportement « nominal » du programme, et le code dédié au traitement d'erreur.
2. Cette syntaxe permet de classifier clairement le code dédié à la gestion de chaque type d'erreur. De même, le code dédié à un type particulier d'erreur est factorisé à un seul endroit (alors que cette erreur peut survenir à plusieurs endroits), évitant ainsi la duplication.

Pas clair ? Prenons un autre exemple. Considérons maintenant le constructeur de notre classe `Pangolin`. Pas simple de construire un pangolin. Plein de choses peuvent se passer : le nombre d'écailles peut être incorrect, le nom peut être déjà pris (chaque pangolin a un nom unique au monde), le nombre maximal de pangolins vivants peut être atteint (oui, la population est limitée...).

```
class NbEcaillesIncorrectException extends Exception {
    public NbEcaillesIncorrectException(String message) {
        super(message);
    }
}

class NbMaxPangolinException extends Exception {
    public NbMaxPangolinException() {
        super("Le nombre maximal de pangolins est atteint...");
    }
}

class NommageException extends Exception {
```

```
    public NommageException(String message) {
        super(message);
    }
}

public class Pangolin {
    private String nom;
    private int nbEcailles;
    private static java.util.Set<String> noms = new
java.util.HashSet<String>();
    private static final int NOMBRE_MAX_PANGOLINS = 42; // Population
mondiale de pangolins

    public Pangolin(String nom, int nbEcailles) {
        try {
            if (Pangolin.noms.size() >= Pangolin.NOMBRE_MAX_PANGOLINS) {
                throw new NbMaxPangolinException();
            }
            if (nbEcailles < 0) {
                throw new NbEcaillesIncorrectException("Nombre d'écailles
négatif !");
            }
            if (nbEcailles == 0) {
                throw new NbEcaillesIncorrectException("Le pangolin est nu.
Il va avoir froid !");
            }
            if (nbEcailles == 13) {
                throw new NbEcaillesIncorrectException("13 épines sur un
pangolin, ça porte malheur...");
            }
            this.nbEcailles = nbEcailles;
            if (Pangolin.noms.contains(nom)) {
                throw new NommageException("Ce nom est déjà pris...");
            }
            if (nom.equals("Kikoolol38")) {
                throw new NommageException("Quel nom ridicule. Faut pas
charrier non plus...");
            }
            this.nom = nom;
            Pangolin.noms.add(nom);
            // Ici on peut mettre plein d'autre code à exécuter
            // pour créer un pangolin...
            // ...
        } catch (NommageException e) {
            System.out.println(e); // On ne fait rien d'autre que
d'afficher l'exception.
        } catch (NbEcaillesIncorrectException e) {
            System.out.println(e); // On ne fait rien d'autre que
d'afficher l'exception.
        } catch (NbMaxPangolinException e) {
            e.printStackTrace();
            System.exit(1); // Gros problème : du coup, on quitte sur une
erreur.
        }
    }
}
```

Dans l'exemple ci-dessus, nous voyons donc que trois types d'exception peuvent survenir lors de la création d'un pangolin : `NbMaxPangolinException`, `NommageException` et `NbEcaillesIncorrectException`. Chaque type d'exception peut se produire à des endroits différents, pour des raisons différentes, mais le gestionnaire dédié à un type d'exception exécutera toujours la même chose, quelle que soit la cause associée à la levée de l'exception.

Dans un bloc `catch`, c'est le type de l'exception passée en paramètre qui déterminera si le gestionnaire est exécuté ou pas. Si l'exception lancée dans le bloc `try` correspond au type du bloc `catch`, alors c'est le code de ce gestionnaire qui sera exécuté.

Rattraper plusieurs exceptions à la fois

Dans l'exemple ci-dessus, vous avez pu remarquer que le traitement associé à `NommageException` et `NbEcaillesIncorrectException` était le même. Est-il possible de factoriser ce code ? Oui, il est possible de le faire depuis Java 7, en utilisant la syntaxe suivante :

```
catch (NommageException | NbEcaillesIncorrectException e) {
    e.printStackTrace(); // On ne fait rien d'autre que d'afficher la pile d'appel
}
```

Gestionnaires et hiérarchie d'exceptions

Je vous ai dit plus haut que si l'exception lancée dans le bloc `try` correspondait au type du bloc `catch`, alors c'est le code de ce gestionnaire qui serait exécuté. En fait, c'est légèrement plus compliqué que cela : pour qu'un bloc `catch` soit exécuté, il faut que l'exception lancée dans le bloc `try` ait un type **compatible** avec le type spécifié dans la clause `catch`. Ici, « compatible » signifie « être du même type ou n'importe quel sous-type ». Prenons un exemple.

Considérons la hiérarchie d'exceptions ci-dessous :

Le code suivant illustre différents cas de figure d'exceptions pouvant être rattrapées par différents gestionnaires.

```
try {
    // Ici du code qui peut lancer plusieurs types d'exception...
} catch (NommageException e) {
    // Ici, tout ce qui est de type NommageException sera traité
} catch (NbMaxPangolinException e) {
    // Ici, tout ce qui est de type NbMaxPangolinException sera traité
} catch (PangolinException e) {
    // Ici, tout ce qui est de type PangolinException sera traité
    // donc en particulier NbEcaillesIncorrectException, mais pas
    // NbMaxPangolinException, qui a déjà été traité plus haut.
} catch (Exception e) {
    // Le gestionnaire attrape-tout. Toutes les exceptions
    // qui n'ont pas été rattrapées plus haut (même les exceptions
    // hors-contrôle d'un sous-type de RuntimeException) seront
    // traitées ici
}
```

Note importante

Une exception ne peut être rattrapée que par un seul gestionnaire d'exception. Techniquement, ce sera le premier gestionnaire ayant un type compatible avec le type de l'exception. Attention donc à l'ordre de déclaration des gestionnaires. Dans l'exemple ci-dessus, si le gestionnaire `catch (PangolinException e)` avait été placé avant `catch`

(NbMaxPangolinException e), il aurait rattrapé également toutes les exceptions de type NbMaxPangolinException, rendant caduque la clause catch (NbMaxPangolinException e). Dans les faits, le compilateur vous le signalera et refusera de compiler.

Clause *finally*

Il est parfois utile de spécifier du code qui soit exécuté quoi qu'il se passe : qu'il y ait une exception levée ou pas, et qu'elle soit traitée localement ou propagée à la méthode appelante. Ce peut être par exemple le cas si vous avez besoin de libérer une ressource (un fichier par exemple) avant de traiter l'erreur ou de quitter le programme sur un message d'insulte.

C'est le rôle de la clause `finally`, toujours placée après le dernier bloc `catch` :

```
try {
    // Ici du code qui peut lancer plusieurs types d'exception...
} catch (...) {
    ...
} catch (...) {
    ...
} finally {
    // Ici du code qui sera exécuté dans tous les cas
}
```

Déléguer le traitement de l'erreur : *throws*

Jusqu'ici nous avons vu comment traiter localement une erreur. Parfois, ce n'est pas possible ou pas souhaitable. Dans l'exemple précédent, après tout, qui est responsable si un pangolin est initialisé avec un nombre négatif d'écailles ? Il peut sembler naturel de dire que c'est de la responsabilité de la méthode appelante. Dans ce cas, il faut que l'exception se **propage** dans la pile d'appels de méthode, afin de déléguer le traitement de cette erreur à la méthode appelante. En gros, vous dites à la méthode appelante : « écoute ma vieille [pardonnez cette familiarité simplement destinée à vous tenir en haleine], tu as fait n'importe quoi avec ce paramètre, tu assumes. ». Mais comme nous l'avons dit ci-avant, vous ne pouvez dire ça que si vous avez prévenu avant. Prévenir, c'est simplement dire que la méthode peut lever une exception de tel type (et préciser dans la documentation dans quels cas cela arrive). Cela se fait avec le mot-clef `throws`.

Reprenons notre classe `Pangolin` :

```
public class Pangolin {
    private String nom;
    private int nbEcailles;
    private static java.util.Set<String> noms = new
java.util.HashSet<String>();
    private static final int NOMBRE_MAX_PANGOLINS = 42; // Population
mondiale de pangolins

    public Pangolin(String nom, int nbEcailles) throws
NbEcaillesIncorrectException, NommageException {
```

```
        try {
            // Le reste du code ne change pas
        } catch (NbMaxPangolinException e) {
            e.printStackTrace();
            System.exit(1); // Gros problème : du coup, on quitte sur une
erreur.
        }
    }
}
```

Dans cette nouvelle version, si le code à l'intérieur du bloc `try` peut encore lever des exceptions de type `NbEcaillesIncorrectException` et `NommageException`, ces exceptions ne seront récupérées par aucun gestionnaire d'exception, contrairement à la version précédente (NB : le fait que ce code soit à l'intérieur d'un bloc `try` ne change rien ici). Pour que le code compile, il est donc indispensable de déclarer ces deux types d'exception dans la signature de la méthode avec `throws`.

La responsabilité du traitement de ces erreurs sera donc transférée à la méthode appelante. Par exemple :

```
public static void main(String[] args) throws NbEcaillesIncorrectException,
NommageException {
    new Pangolin("Gérard", -5);
}
```

Ici, l'utilisateur a choisi de ne pas traiter localement les erreurs, donc, vu que la contrainte *catch or specify* s'applique ici, les deux exceptions doivent être déclarées dans la signature de la méthode. Les plus alertes d'entre vous ont dû voir qu'il s'agissait de la méthode `main` : ce qui se passera en pratique dans ce cas sera donc que le programme quittera sur une erreur, en affichant l'exception et sa pile d'appel.

Attention au gestionnaire attrape-tout

Vous l'avez compris, la contrainte *catch or specify* vous oblige à traiter localement toute exception sous contrôle pouvant être lancée, ou à la déclarer dans la signature (ce qui vous obligera à la traiter dans la méthode appelante). Si vous avez du code qui peut lancer plusieurs exceptions différentes, c'est pénible et il peut être tentant de « museler » ces exceptions en utilisant un bloc `catch` attrape-tout :

```
try {
    // Ici du code qui peut lancer plein d'exceptions
} catch (Exception e) {}
```

Bel exemple de stratégie consistant à dissimuler les erreurs sous le tapis. Dans le meilleur des cas il ne se passe rien et tant mieux. Mais s'il y a une erreur, le code n'est pas exécuté, l'erreur n'est pas affichée (car le gestionnaire d'exception la rattrape et ne fait rien) et provoquera probablement des problèmes à d'autres endroits du code qui n'ont rien à voir avec cette méthode. Vous pouvez passer des jours à comprendre ce qui se passe. Avec un peu de chance, cela concerne en plus un code que

vous avez écrit il y a six mois et vous avez oublié cette petite entorse à toutes les règles de déontologie informatique.

Moralité : ne le faites pas. Le dernier stagiaire qui a tenté a fini pendu par les pieds au vidéoprojecteur de la salle de réunion.

Exemple complet

Pour conclure cette fiche d'exception, voici un petit exercice.

Considérons la hiérarchie d'exceptions suivante :

Considérons maintenant le code suivant :

```
public class TestException {
    private static final boolean PANGOLIN_EXCEPTION = false;
    private static final boolean NOMMAGE_EXCEPTION = false;
    private static final boolean CANARD_EXCEPTION = false;
    private static final boolean NBECAILLESINCORRECT_EXCEPTION = false;
    private static final boolean NBMAXPANGOLIN_EXCEPTION = false;
    private static final boolean ARITHMETIC_EXCEPTION = false;
    private static final boolean ILLEGAL_ARGUMENT_EXCEPTION = false;

    public static void m1() throws CanardException {
        try {
            System.out.println("Début de m1()");
            m2();
            System.out.println("Fin de m1()");
        } catch (PangolinException e) {
            System.out.println("Exception Pangolin (de m1()).");
        }
    }

    public static void m2() throws PangolinException, CanardException {
        try {
            System.out.println("Début de m2()");
            m3();
            System.out.println("Fin de m2()");
        } catch (NbMaxPangolinException e) {
            System.out.println("Exception nombre max pangolin (de m2()).");
            throw new IllegalArgumentException();
        } finally {
            System.out.println("Finally de m2()");
        }
        System.out.println("Fin de m2() après le try/catch/finally");
    }

    public static void m3() throws PangolinException, CanardException {
        try {
            System.out.println("Début de m3()");
            if (PANGOLIN_EXCEPTION)
                throw new PangolinException();
            if (NOMMAGE_EXCEPTION)
                throw new NommageException();
            if (CANARD_EXCEPTION)
                throw new CanardException();
            if (NBECAILLESINCORRECT_EXCEPTION)
                throw new NbEcaillesIncorrectException();
            if (NBMAXPANGOLIN_EXCEPTION)
                throw new NbMaxPangolinException();
```

```
            if (ARITHMETIC_EXCEPTION)
                throw new ArithmeticException();
            if (ILLEGAL_ARGUMENT_EXCEPTION)
                throw new IllegalArgumentException();
            System.out.println("Fin de m3()");
        } catch (NommageException e) {
            System.out.println("Exception Nommage (de m3()).");
        }
    }

    public static void main(String[] args) throws CanardException {
        System.out.println("Début de la méthode principale");
        m1();
        System.out.println("Fin de la méthode principale");
    }
}
```

Nous allons maintenant essayer de déterminer ce qui se passe selon les valeurs des attributs booléens déclenchant ou non l'envoi des exceptions.